In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
```

Step 1 — Data Cleaning

Step 1 — Dataset Load

In [3]:
```
df = pd.read_csv(r"C:\Users\Sonal.DESKTOP-0RFE5RA\OneDrive\Desktop\Data_Analyst\
df.head()
```

Out[3]:

	Order_ID	Order_Date	City	State	Restaurant	Dish_Name	Category	Cust
0	1001	01-01-2024 00:00	Hyderabad	Telangana	Urban Bites	Veg Burger	Veg	
1	1002	01-01-2024 01:00	Kolkata	Telangana	Tandoori Treat	Chicken Roll	Veg	
2	1003	01-01-2024 02:00	Bangalore	Telangana	Food Corner	Fish Curry	Veg	
3	1004	01-01-2024 03:00	Kolkata	Telangana	Spice Hub	Paneer Butter Masala	Veg	
4	1005	01-01-2024 04:00	Kolkata	West Bengal	Royal Kitchen	Chicken Roll	Non-Veg	

Step 2 — Dataset Structure Check

In [4]:
```
df.shape
```

Out[4]:
```
(1000, 12)
```

Check column names

In [6]:
```
df.columns
```

Out[6]:
```
Index(['Order_ID', 'Order_Date', 'City', 'State', 'Restaurant', 'Dish_Name',
       'Category', 'Customer_ID', 'Quantity', 'Price (INR)', 'Rating',
       'Delivery_Time_Min'],
      dtype='object')
```

Data types

In [7]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 12 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Order_ID           1000 non-null   int64
 1   Order_Date         1000 non-null   object
 2   City               1000 non-null   object
 3   State              1000 non-null   object
 4   Restaurant         1000 non-null   object
 5   Dish_Name          1000 non-null   object
 6   Category           1000 non-null   object
 7   Customer_ID        1000 non-null   int64
 8   Quantity           1000 non-null   int64
 9   Price (INR)        1000 non-null   int64
 10  Rating             1000 non-null   float64
 11  Delivery_Time_Min  1000 non-null   int64
dtypes: float64(1), int64(5), object(6)
memory usage: 93.9+ KB
```

Convert Date

In [6]:
```
df["Order_Date"] = pd.to_datetime(df["Order_Date"], format = "%d-%m-%Y %H:%M")
```

Step 3 — New Columns Create

Create Time Columns

In [7]:
```
df["Year"] = df["Order_Date"].dt.year
df["Month"] = df["Order_Date"].dt.month_name()
df["Day"] = df["Order_Date"].dt.day_name()
df["Hour"] = df["Order_Date"].dt.hour
```

Step 4 — Basic Statistics

In [8]:
```
df.describe()
```

Out[8]:

	Order_ID	Order_Date	Customer_ID	Quantity	Price (INR)	Rat
count	1000.000000	1000	1000.000000	1000.000000	1000.00000	1000.000
mean	1500.500000	2024-01-21 19:29:59.999999744	149.716000	2.504000	294.12300	4.243
min	1001.000000	2024-01-01 00:00:00	100.000000	1.000000	100.00000	3.500
25%	1250.750000	2024-01-11 09:45:00	124.000000	1.750000	193.75000	3.900
50%	1500.500000	2024-01-21 19:30:00	151.000000	3.000000	288.00000	4.200
75%	1750.250000	2024-02-01 05:15:00	175.000000	3.000000	395.00000	4.600
max	2000.000000	2024-02-11 15:00:00	199.000000	4.000000	499.00000	5.000
std	288.819436	NaN	28.640384	1.115001	116.22927	0.437

Step 5 — Data Cleaning

Missing values

In [9]:
```
df.isnull().sum()
```

Out[9]:
```
Order_ID            0
Order_Date          0
City                0
State               0
Restaurant          0
Dish_Name           0
Category            0
Customer_ID         0
Quantity            0
Price (INR)         0
Rating              0
Delivery_Time_Min   0
Year                0
Month               0
Day                 0
Hour                0
dtype: int64
```

Remove duplicates

In [11]:
```
df.drop_duplicates(inplace=True)
```

Date format convert

In [13]:
```
df["Order_Date"] = pd.to_datetime(df["Order_Date"])
```

30 Real Data Analyst Questions

Sales Analysis

Total sales kitni hai

In [15]:
```
df["Price (INR)"].sum()
```

Out[15]:
```
np.int64(294123)
```

In [ ]:
```
Profit Column Create karo
```

In [16]:
```
df["Profit"] = df["Price (INR)"] * df["Quantity"] * 0.03
```

In [22]:
```
df.head()
```

Out[22]:

	Order_ID	Order_Date	City	State	Restaurant	Dish_Name	Category	Cust
0	1001	2024-01-01 00:00:00	Hyderabad	Telangana	Urban Bites	Veg Burger	Veg	
1	1002	2024-01-01 01:00:00	Kolkata	Telangana	Tandoori Treat	Chicken Roll	Veg	
2	1003	2024-01-01 02:00:00	Bangalore	Telangana	Food Corner	Fish Curry	Veg	
3	1004	2024-01-01 03:00:00	Kolkata	Telangana	Spice Hub	Paneer Butter Masala	Veg	
4	1005	2024-01-01 04:00:00	Kolkata	West Bengal	Royal Kitchen	Chicken Roll	Non-Veg	

Total profit kitna hai

In [18]:
```
df["Profit"].sum()
```

Out[18]:
```
np.float64(22124.67)
```

Average order value

In [19]:
```
df["Price (INR)"].mean()
```

Out[19]:
```
np.float64(294.123)
```

Total Revenue

In [20]:
```
df["Revenue"] = df["Price (INR)"] * df["Quantity"]
df["Revenue"].sum()
```

Out[20]:
```
np.int64(737489)
```

Total Orders

In [21]:
```
df["Order_ID"].count()
```

Out[21]:
```
np.int64(1000)
```

Product Analysis

Top selling product

In [30]:
```
df.groupby("Dish_Name")["Revenue"].sum().sort_values(ascending=False)
```

Out[30]:
```
Dish_Name
Chicken Roll            113937
Paneer Butter Masala    110837
Veg Burger              109239
Masala Dosa             107440
Chicken Biryani         105725
Fish Curry               96929
Veg Pizza                93382
Name: Revenue, dtype: int64
```

Category wise sales

In [25]:
```
df.groupby("Category")["Revenue"].sum()
```

Out[25]:
```
Category
Non-Veg    360381
Veg        377108
Name: Revenue, dtype: int64
```

Category wise profit

In [26]:
```
df.groupby("Category")["Profit"].sum()
```

Out[26]:
```
Category
Non-Veg    10811.43
Veg        11313.24
Name: Profit, dtype: float64
```

Region Analysis

Revenue by City

In [29]:
```
df.groupby("City")["Revenue"].sum()
```

Out[29]:
```
City
Bangalore    138362
Delhi        143650
Hyderabad    158540
Kolkata      139646
Mumbai       157291
Name: Revenue, dtype: int64
```

Revenue by State

In [31]:
```
df.groupby("State")["Revenue"].sum()
```

Out[31]:
```
State
Delhi          153319
Karnataka      132592
Maharashtra    161055
Telangana      129920
West Bengal    160603
Name: Revenue, dtype: int64
```

Customer Analysis

In [32]:
```
df.groupby("Customer_ID")["Revenue"].sum().sort_values(ascending=False)
```

Out[32]:
```
Customer_ID
112    14618
151    13814
139    13545
162    12199
198    12153
       ...
177     2583
199     2027
150     1385
190     1347
132      743
Name: Revenue, Length: 100, dtype: int64
```

Total unique customers

In [33]:
```
df["Customer_ID"].nunique()
```

Out[33]:
```
100
```

Average Order Value

In [34]:
```
df["Revenue"].mean()
```

Out[34]:
```
np.float64(737.489)
```

Time Analysis

Monthly Revenue

In [37]:
```
df.groupby("Month")["Revenue"].sum()
```

Out[37]:
```
Month
February    187138
January     550351
Name: Revenue, dtype: int64
```

Year Revenue

In [38]:
```
df.groupby("Year")["Revenue"].sum()
```

Out[38]:
```
Year
2024    737489
Name: Revenue, dtype: int64
```

Hourly Orders

In [39]:
```
df.groupby("Hour")["Order_ID"].count()
```

Out[39]:
```
Hour
0     42
1     42
2     42
3     42
4     42
5     42
6     42
7     42
8     42
9     42
10    42
11    42
12    42
13    42
14    42
15    42
16    41
17    41
18    41
19    41
20    41
21    41
22    41
23    41
Name: Order_ID, dtype: int64
```

In [40]:
```
df.groupby("Hour")["Revenue"].sum()
```

Out[40]:
```
Hour
0     36099
1     29187
2     27663
3     31260
4     31939
5     29734
6     30356
7     32274
8     35623
9     28106
10    28068
11    30062
12    33585
13    29774
14    25775
15    30197
16    32781
17    31499
18    26519
19    30885
20    30603
21    34077
22    32016
23    29407
Name: Revenue, dtype: int64
```

Rating Analysis

In [ ]:
```
Average Rating
```

In [41]:
```
df["Rating"].mean()
```

Out[41]:
```
np.float64(4.2432)
```

Rating by Dish

In [43]:
```
df.groupby("Dish_Name")["Rating"].mean()
```

Out[43]:
```
Dish_Name
Chicken Biryani         4.267164
Chicken Roll            4.225806
Fish Curry              4.305926
Masala Dosa             4.266447
Paneer Butter Masala    4.244366
Veg Burger              4.173333
Veg Pizza               4.226515
Name: Rating, dtype: float64
```

Delivery Performance

Average Delivery Time

In [44]:
```
df["Delivery_Time_Min"].mean()
```

Out[44]: np.float64(39.304)

Delivery Time by City

In [46]:
```
df.groupby("City")["Delivery_Time_Min"].mean()
```

Out[46]:
```
City
Bangalore    39.094737
Delhi        38.394737
Hyderabad    40.334951
Kolkata      39.122549
Mumbai       39.480952
Name: Delivery_Time_Min, dtype: float64
```

Discount Analysis

Average discount

In [49]:
```
import numpy as np

df["Discount (%)"] = np.random.randint(5, 30, size=len(df))   # 5%–30%
```

Total Discount diya

In [54]:
```
df["Discount (%)"].mean()
```

Out[54]: np.float64(16.63)

In [52]:
```
print(df.columns)
```

```
Index(['Order_ID', 'Order_Date', 'City', 'State', 'Restaurant', 'Dish_Name',
       'Category', 'Customer_ID', 'Quantity', 'Price (INR)', 'Rating',
       'Delivery_Time_Min', 'Year', 'Month', 'Day', 'Hour', 'Profit',
       'Revenue', 'Discount (%)'],
      dtype='object')
```

In [55]:
```
df.groupby("Discount (%)")["Revenue"].mean()
```

Out[55]:
```
Discount (%)
5     728.684211
6     738.600000
7     827.515152
8     687.675676
9     634.928571
10    827.431818
11    730.632653
12    734.700000
13    789.220000
14    710.530612
15    895.045455
16    670.787879
17    745.250000
18    661.756757
19    717.565217
20    678.100000
21    757.512195
22    715.380952
23    772.263158
24    678.088889
25    729.333333
26    729.032258
27    731.285714
28    748.727273
29    768.789474
Name: Revenue, dtype: float64
```

Payment Analysis

Random Payment Method

In [56]:
```
import numpy as np

payment_methods = ["Cash", "UPI", "Credit Card", "Debit Card", "Wallet"]

df["Payment Method"] = np.random.choice(payment_methods, size=len(df))
```

Kitne orders kis method se aaye

In [57]:
```
df["Payment Method"].value_counts()
```

Out[57]:
```
Payment Method
Cash           214
Debit Card     205
Credit Card    199
Wallet         198
UPI            184
Name: count, dtype: int64
```

Revenue by Payment Method

In [59]:
```
df.groupby("Payment Method")["Revenue"].sum()
```

Out[59]:
```
Payment Method
Cash           156377
Credit Card    143314
Debit Card     147088
UPI            137722
Wallet         152988
Name: Revenue, dtype: int64
```

Average Order Value

In [60]:
```
df.groupby("Payment Method")["Revenue"].mean()
```

Out[60]:
```
Payment Method
Cash           730.733645
Credit Card    720.170854
Debit Card     717.502439
UPI            748.489130
Wallet         772.666667
Name: Revenue, dtype: float64
```

Total Orders by Payment

In [61]:
```
df.groupby("Payment Method")["Order_ID"].count()
```

Out[61]:
```
Payment Method
Cash           214
Credit Card    199
Debit Card     205
UPI            184
Wallet         198
Name: Order_ID, dtype: int64
```

Food Category Analysis

In [62]:
```
df.groupby("Category")["Revenue"].sum()
```

Out[62]:
```
Category
Non-Veg    360381
Veg        377108
Name: Revenue, dtype: int64
```

Order Pattern

In [63]:
```
df.groupby("Day")["Order_ID"].count()
```

Out[63]:
```
Day
Friday       144
Monday       144
Saturday     144
Sunday       136
Thursday     144
Tuesday      144
Wednesday    144
Name: Order_ID, dtype: int64
```

Price Analysis

In [65]:
```
df["Price (INR)"].describe()
```

Out[65]:
```
count    1000.00000
mean      294.12300
std       116.22927
min       100.00000
25%       193.75000
50%       288.00000
75%       395.00000
max       499.00000
Name: Price (INR), dtype: float64
```

Customer Segmentation

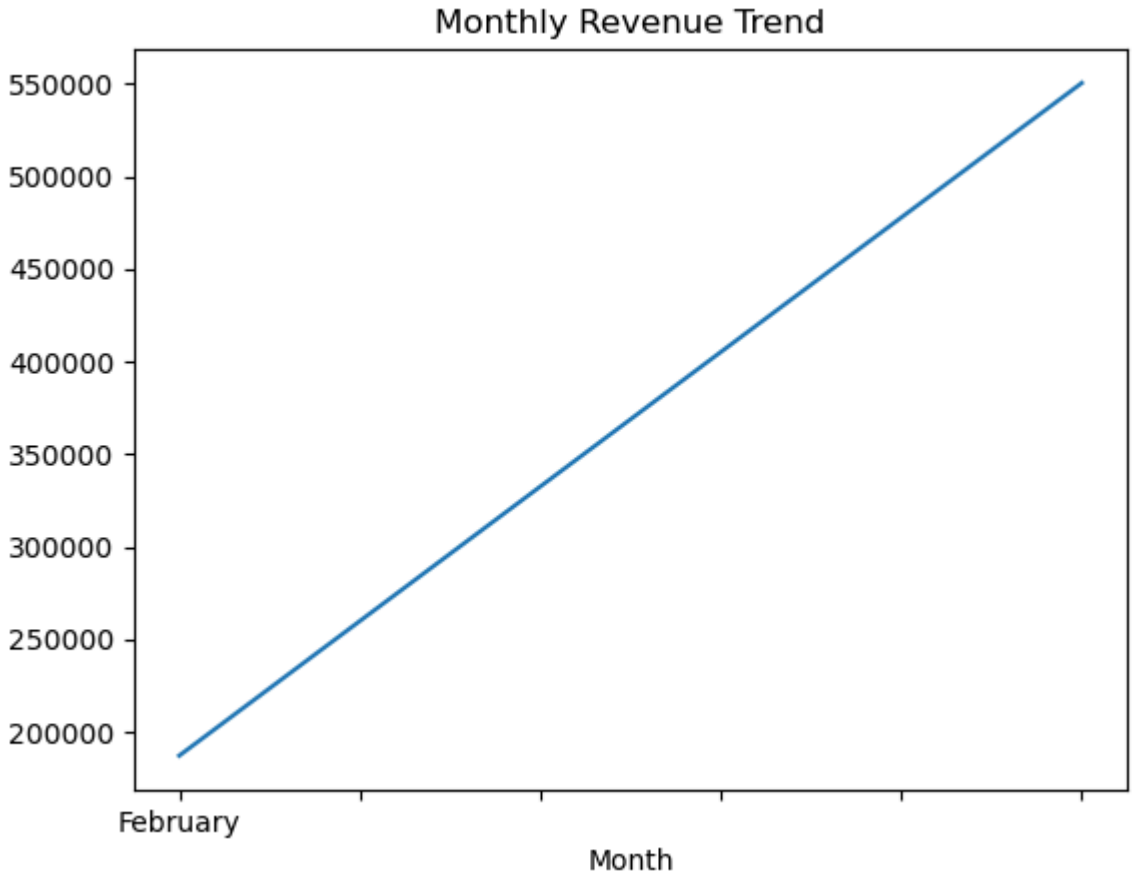
In [66]:
```
df.groupby("Customer_ID")["Revenue"].sum()
```

Out[66]:
```
Customer_ID
100     5951
101     8166
102     5116
103    10348
104     6280
       ...
195     7288
196     5058
197     4885
198    12153
199     2027
Name: Revenue, Length: 100, dtype: int64
```

Visualization Charts

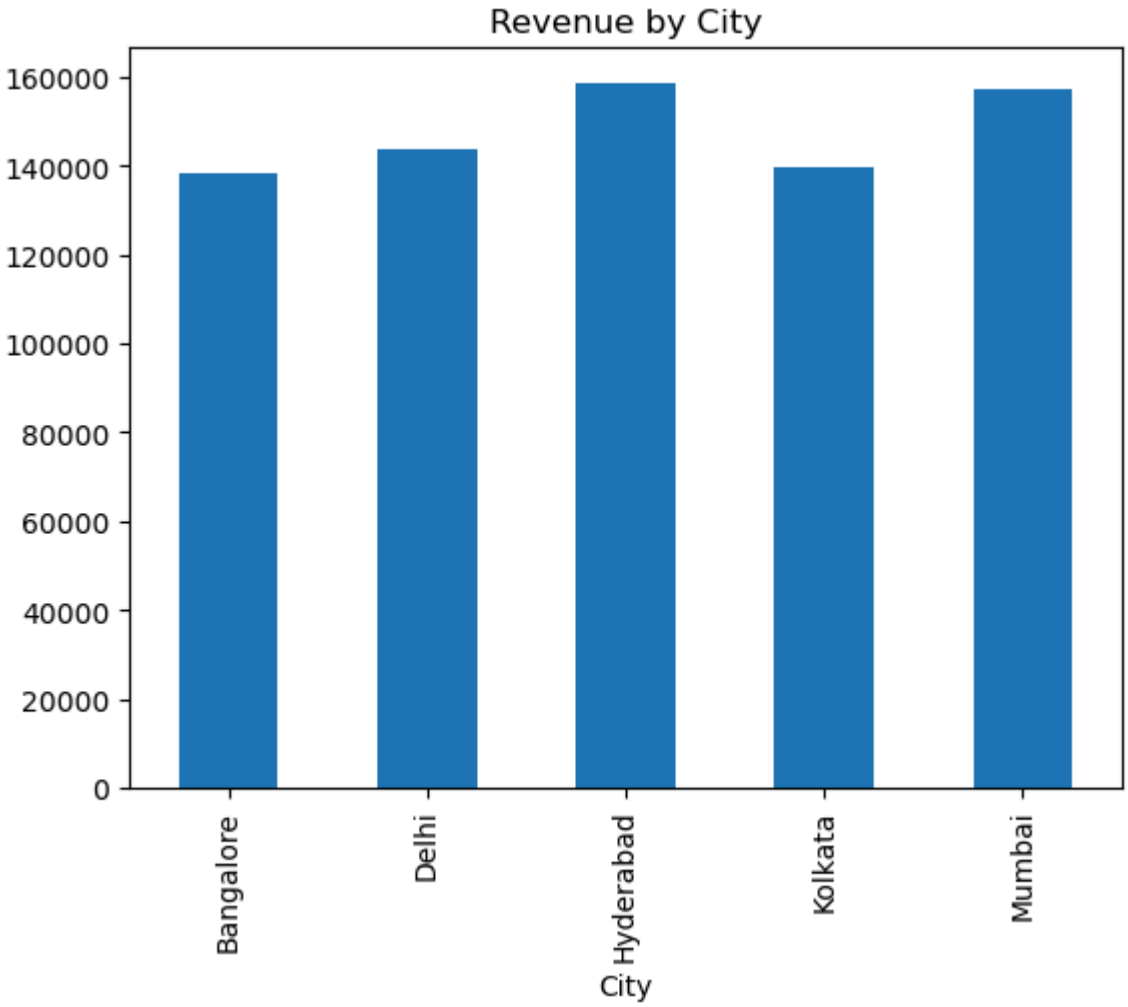
Monthly Revenue Trend

In [67]:
```
import matplotlib.pyplot as plt

monthly = df.groupby("Month")["Revenue"].sum()
monthly.plot(kind = "line")
plt.title("Monthly Revenue Trend")
plt.show()
```



City Revenue

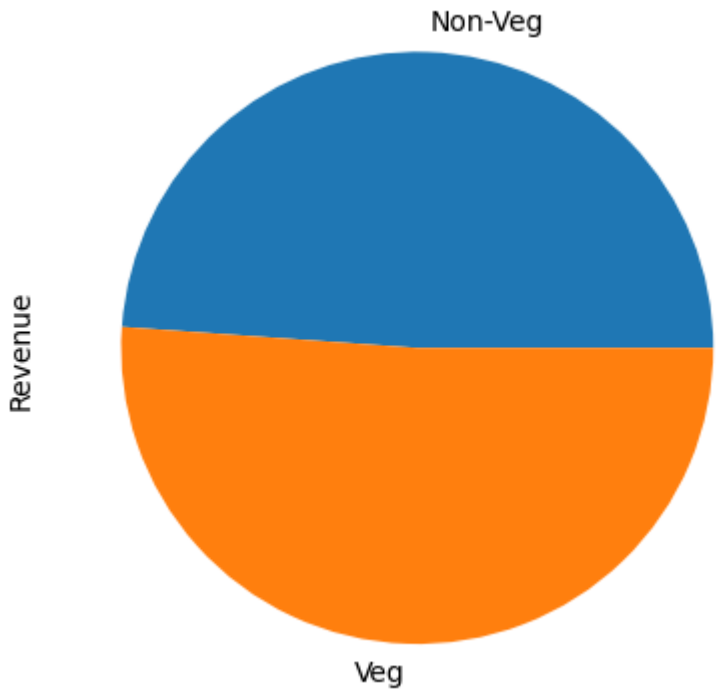
In [70]:
```
city = df.groupby("City")["Revenue"].sum()

city.plot(kind="bar")

plt.title("Revenue by City")
plt.show()
```



Category Contribution

In [71]:
```
df.groupby("Category")["Revenue"].sum().plot(kind="pie")
```

Out[71]:
```
<Axes: ylabel='Revenue'>
```



In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: